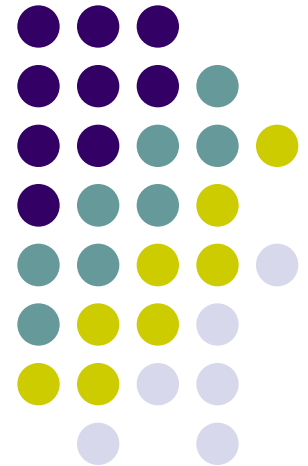
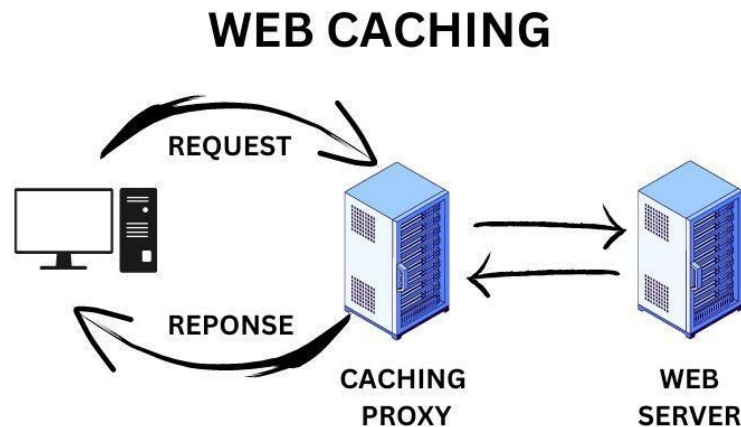


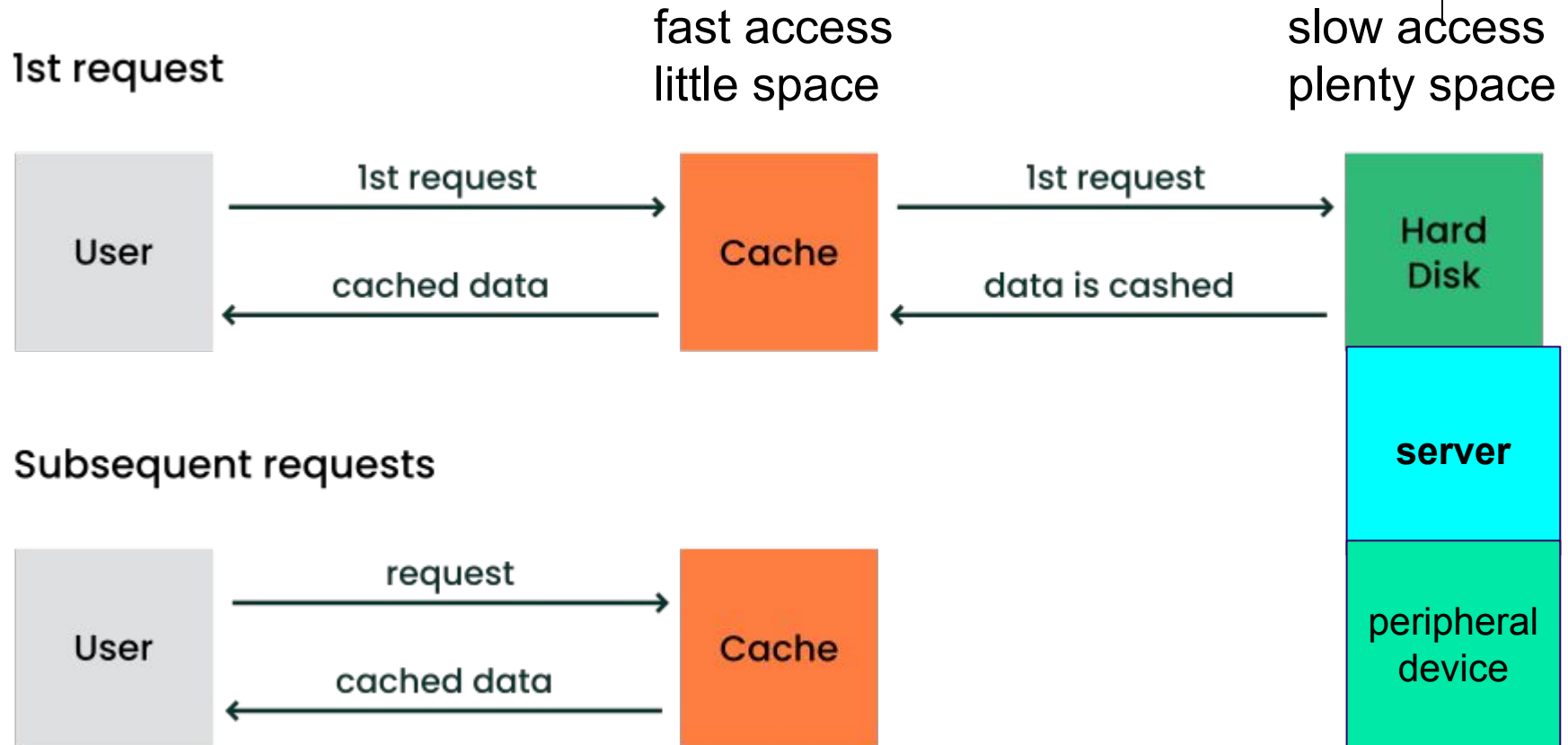
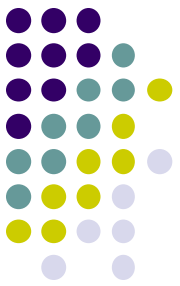
Uso de Caches



Motivação

- Devido a grande disparidade de tempos de acesso ao disco e a outros dispositivos, é importante manter dados e instruções “sendo usadas” prontos para uso em uma memória rápida (cache)
- Isso é muito comum para discos e acesso remoto rede (caching proxy), mas até entre a CPU e a memória RAM tem vários níveis de cache.
- E na memória virtual vimos também que a MMU usa uma Translation-Lookaside Buffer, que nada mais é do que um Cache associativo.





A diferença entre o espalhamento dos endereços (keys) de acesso e o tamanho do cache determinam tudo.

Caching



É uma forma de acelerar acesso a dados que levariam muito tempo para serem acessados.

⇒ banco de dados, sistema de arquivos, servidor em rede, e até memória principal

É tecnologia de memória de acesso rápido,

- tem tamanho limitado ⇒ não consegue acomodar todos os registros/dados
- só contém uma cópia do registro/dado do repositório persistente ou servidor acessado mais recentemente (ou mais frequentemente)
- é memória volátil ⇒ dados podem ser perdidos

Caching



Em cada acesso, primeiro verifica-se se o registro está no cache:

- se sim (**cache_hit**) $\Rightarrow$ retorna o dado que está lá
- senão (**cache_miss**) $\Rightarrow$ {
 - consulta-se o repositório persistente (BD, servidor)
 - copia-se o dado acessado para o cache}

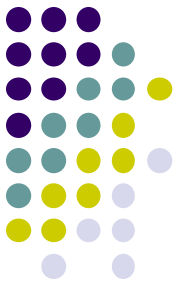
E como fazer quando o cache está cheio?

Como remover itens?

Alternativas:

- sobre escrever qualquer entrada
- usando TTL = Time To Live, remover entradas que já estão muito tempo lá

Cinco padrões + comuns de uso do Cache



Depende do tipo de dado, do grau de consistência exigido pela aplicação, e do padrão de acesso

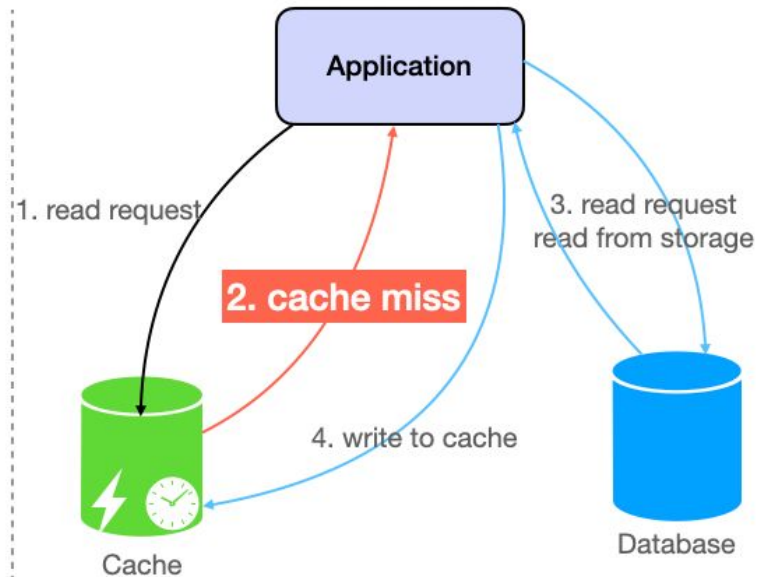
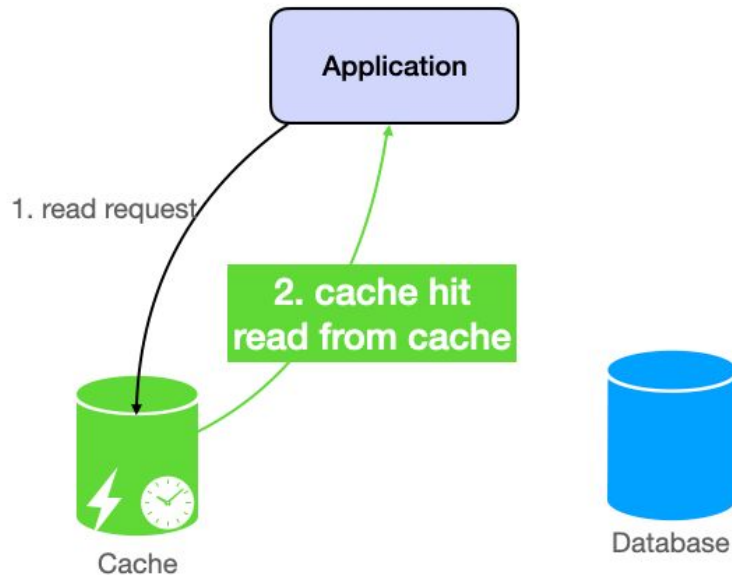
1. Cache-aside (mais comum)
2. Write-through
3. Write-behind (write-back)
4. Event-driven invalidation
5. Stale-while-revalidate

Cache-aside caching



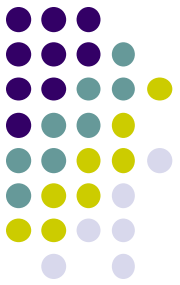
É o mais comum e simples. Aplicação verifica o cache primeiro. Se tem (hit), retorna. Se não tem (miss), busca no banco, salva no cache, retorna.

Problema: entre o banco atualizar e o cache expirar, o dado está *stale*. Se isso importa, precisa de invalidação ativa.

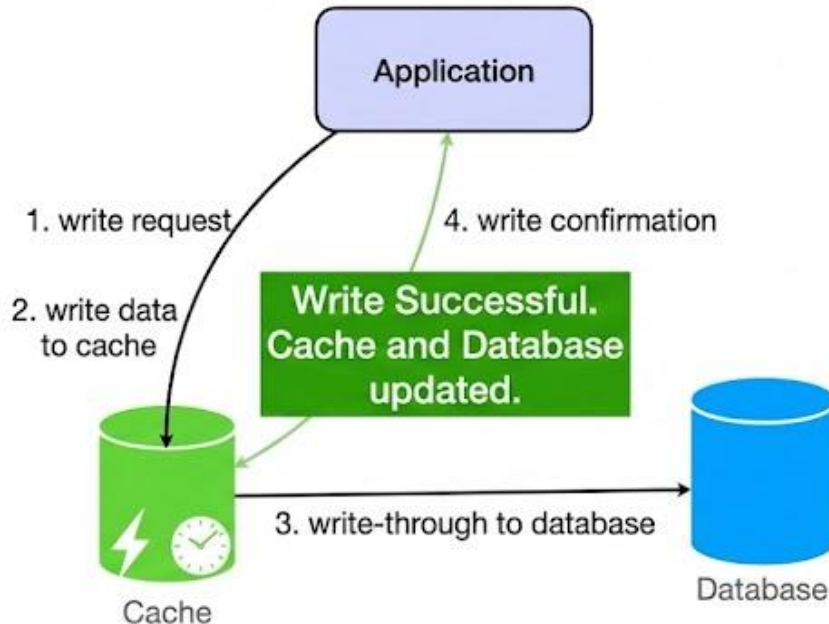


Cache write through

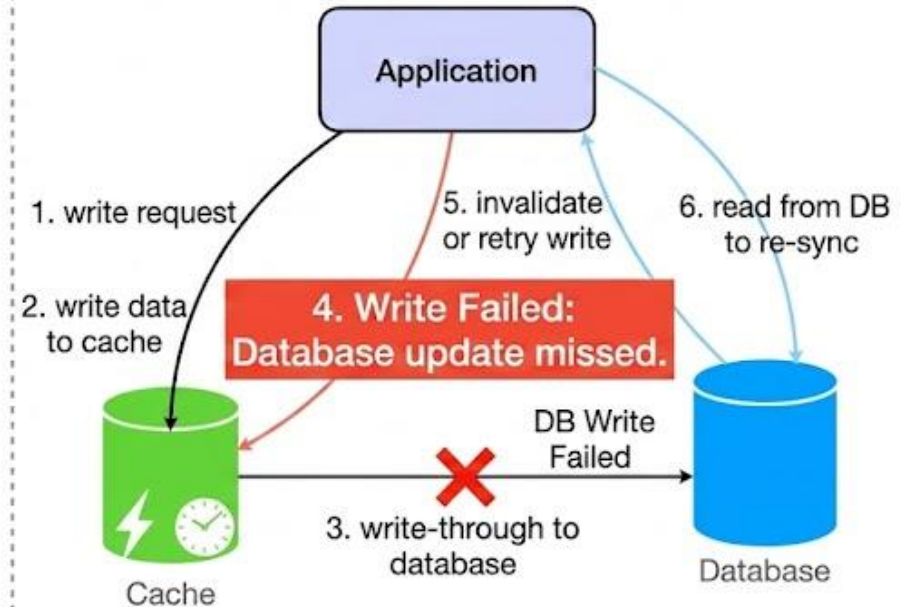
Toda escrita **vai para banco e para cache ao mesmo tempo**. Cache está sempre atualizado.
Custo: toda escrita é mais lenta (duas operações). Vale a pena quando leitura é muito mais frequente que escrita.



Successful Cache Write



Failed Cache Write (Re-sync)



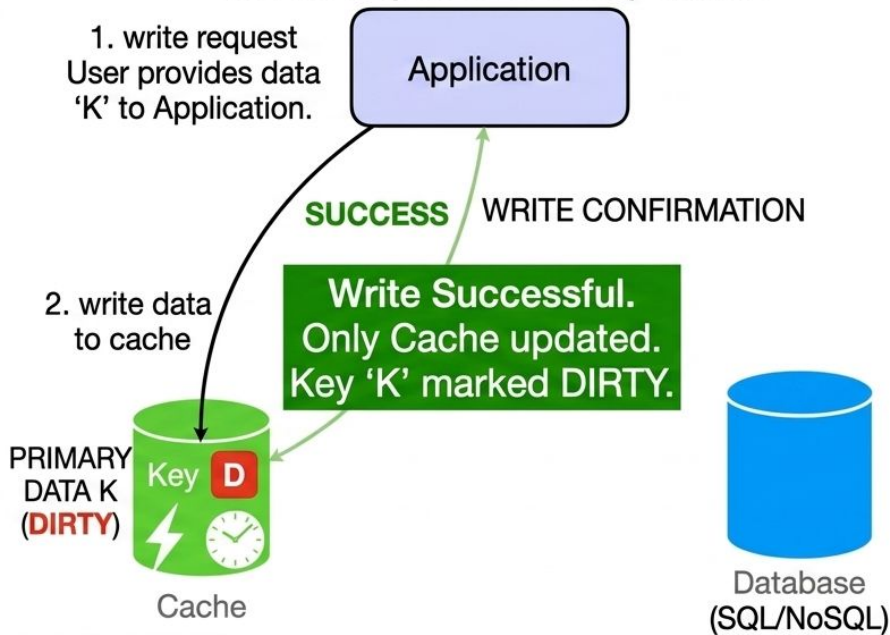
Cache write back (write behind)



Escrita vai primeiro pro cache e é persistida no banco de forma assíncrona.
Latência de escrita mínima.

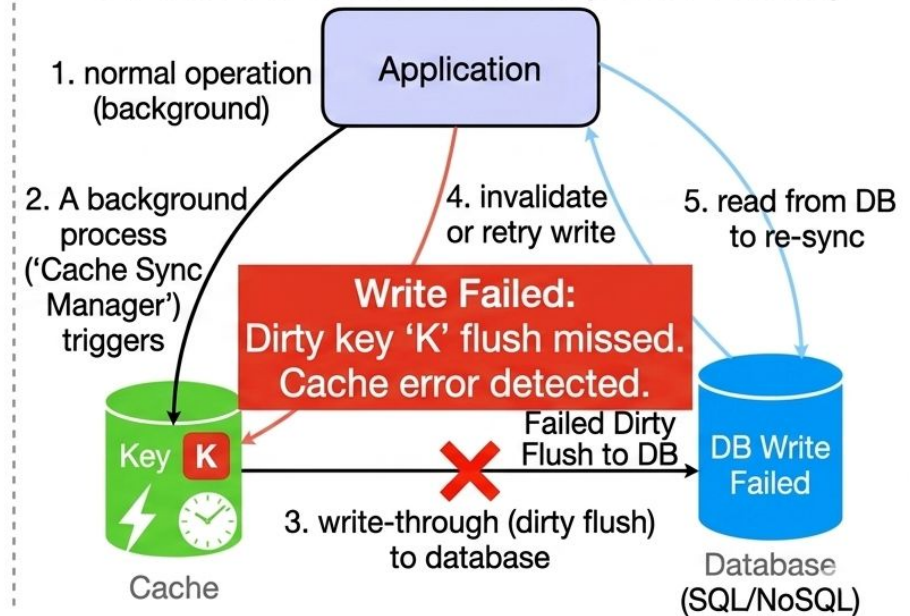
Risco: se o cache cai antes de persistir, perde dados. Uso com cautela!

NORMAL (WRITE-BACK) WRITE

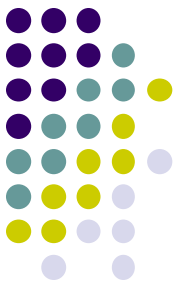


WRITE-BACK CACHING

DB SYNC & CACHE ERROR (DIRTY FLUSH)



Event-driven cache invalidation



Quando o dado muda no banco, um evento é emitido e todos os caches são invalidados ativamente. Cache está sempre fresco sem depender de TTL.
Com Pub/Sub do Redis ou CDC (Change Data Capture) do PostgreSQL.

EVENT-DRIVEN INVALIDATION CACHING

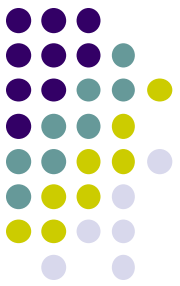
SECTION 1: DATABASE UPDATE & EVENT EMISSION



SECTION 2: CACHE INVALIDATION REACTION



Stale-while-revalidate

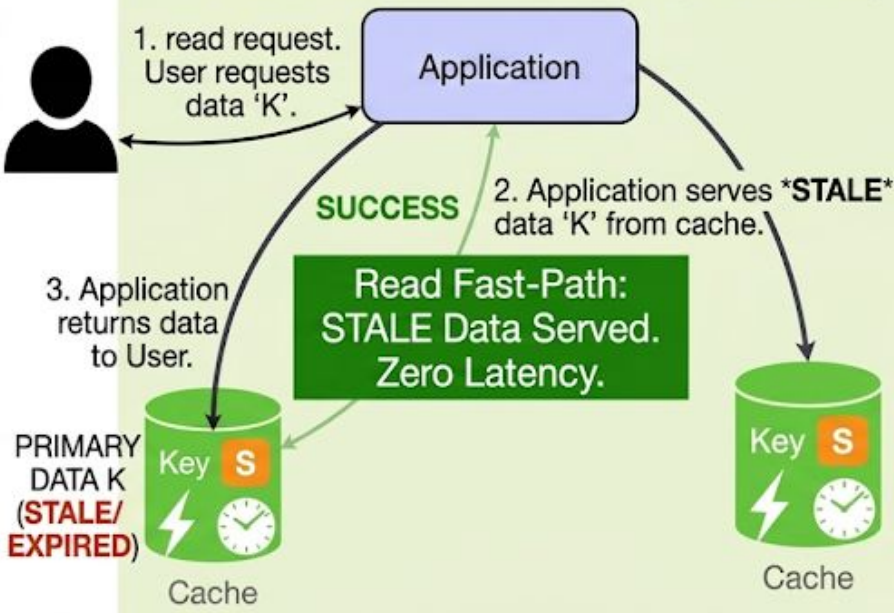


Retorna o dado do cache (mesmo expirado) imediatamente e atualiza em background. Usuário sempre recebe resposta rápida. Dado pode estar 30 segundos atrasado, mas a experiência é instantânea.

Perfeito pra dashboards, listagens e qualquer dado onde **consistência eventual** é aceitável.

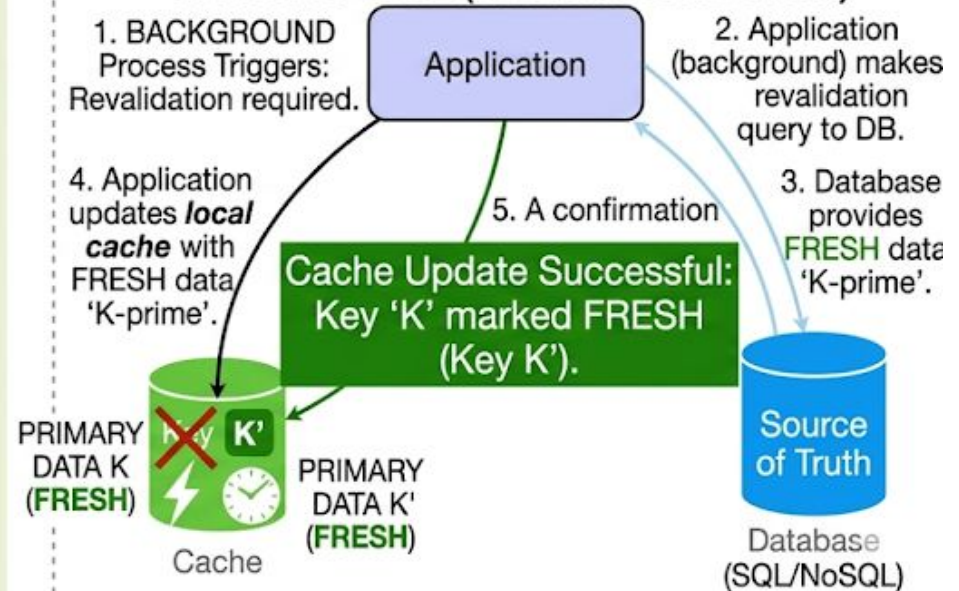
STALE-WHILE-REVALIDATE CACHING

FAST READ OF STALE DATA (FAST-PATH)



recuperação imediata dos dados (expirados) do cache, garantindo uma resposta rápida.

BACKGROUND REVALIDATION & CACHE UPDATE (BACKGROUND-PATH)



um processo em segundo plano revalida o cache. Se a revalidação for bem-sucedida, o cache é atualizado com os dados mais recentes, marcado como “FRESH”, e é retornada uma confirmação

Brincando de Cache



Usando essa estrutura para a entrada do cache

```
#include <time.h>
typedef struct {
    int key;          // Integer in the range 0..999
    int value;        // Integer value
    time_t timestamp; // time when entry was
                        // written/accessed
} CacheEntry;
```

Assuma que o seu banco de dados tem entradas (keys) do intervalo 0..999 e suporta duas operações:

<R, k> : retorna o valor da entrada k

<W(new). K>: atualiza a entrada k com new

Inicialmente todos os valores do banco de dados estão escritos com valores {1..1000}

Acessos Aleatórios a key = {0..999}

```
#define MAX_KEY 999
#define MIN_KEY 0

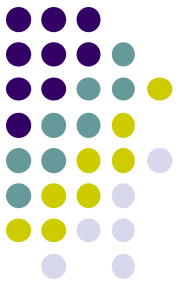
typedef enum {
    READ, WRITE
} OpType;

// Função que gera e exibe N operações aleatórias
void gerar_operacoes(int n) {
    // Inicializa o gerador de números aleatórios com o tempo atual
    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        // Escolhe aleatoriamente entre READ (0) e WRITE (1)
        OpType op = rand() % 2;

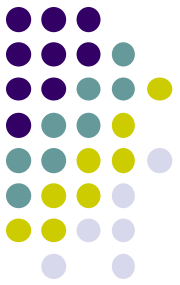
        // Gera key aleatória no intervalo de 0 a 999
        int key = (rand() % (MAX_KEY - MIN_KEY + 1)) + MIN_KEY;

        if (op == READ) {
            // Formato pedido: R, key
            printf("R, %d\n", key);
        } else {
            // Gera um valor inteiro aleatório qualquer (ex: entre 10 e 5000)
            int value = (rand() % 4991) + 10;

            // Formato pedido: W, value, key
            printf("W, %d, %d\n", value, key);
        }
    }
}
```



Consulta ao Cache



```
#include <time.h>
#define CACHE_SIZE 1000
#define CACHE_HIT 1
#define CACHE_MISS 0

// O Cache é um vetor direto de CACHE_SIZE posições
CacheEntry cache[CACHE_SIZE];

// Uso de vetor auxiliar para indicar se entrada possui um dado válido (1) ou está vazio (0)
int is_valid[CACHE_SIZE] = {0};

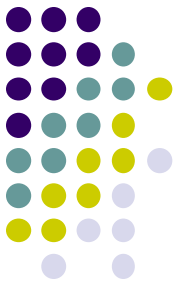
int get(int key, int* out_value) {
    if (key < 0 || key >= CACHE_SIZE) return CACHE_MISS;

    // Verifica se o slot correspondente à chave contém um registro válido
    if (is_valid[key]) {
        // CACHE HIT: Atualiza parâmetro de saída out_value com o valor guardado
        *out_value = cache[key].value;

        // Atualiza o timestamp para registrar o momento do último acesso (útil para políticas
        // como LRU)
        cache[key].timestamp = time(NULL);

        return CACHE_HIT;
    }
    // CACHE MISS: A posição existe no array, mas está vazia/inválida
    return CACHE_MISS;
}
```

Leitura (lenta) do Banco de Dados



```
#define DB_SUCCESS 1
#define DB_ERROR 0
#define DELAY 10
```

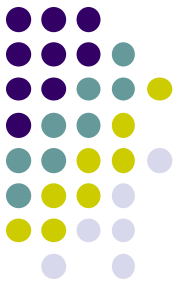
```
int banco_de_dados[DB_SIZE];
```

```
/* Procedimento para simular a LEITURA no Banco de Dados.
 * @return DB_SUCCESS (1) se a leitura foi bem-sucedida, DB_ERROR (0) se a chave for inválida
 */
```

```
int db_read(int key, int* out_value) {
    // Validação de segurança dos limites do banco de dados
    if (key < 0 || key >= 1000) {
        printf("[DB-ERRO] Tentativa de leitura em chave inválida: %d\n", key);
        return DB_ERROR;
    }
    // Copia o valor do banco de dados para o ponteiro de saída
    *out_value = banco_de_dados[key];

    printf("[DB-READ] Lendo do Banco -> Chave: %d, Valor: %d\n", key, *out_value);
    sleep(DELAY);
    return DB_SUCCESS;
}
```

Escrita (lenta) no Banco de Dados

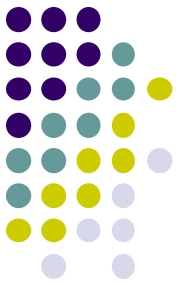


```
/**
 * Procedimento para simular a ESCRITA / ATUALIZAÇÃO no Banco de Dados.
 */
int db_write(int key, int new_value) {
    // Validação de segurança dos limites do banco de dados
    if (key < 0 || key >= 1000) {
        printf("[DB-ERRO] Tentativa de escrita em chave inválida: %d\n", key);
        return DB_ERROR;
    }

    // Atualiza o registro no banco de dados com o novo valor
    banco_de_dados[key] = new_value;
    sleep(DELAY);

    printf("[DB-WRITE] Gravando no Banco -> Chave: %d, Novo Valor: %d\n", key, new_value);
    return DB_SUCCESS;
}
```

Exercícios



- 1) Escreva um programa `log_gen.c` que usa `void gerar_operacoes(int n)` para que ele escreva 10.000 operações em um arquivo `log.txt`
- 2) Escreva um outro programa `main.c` que
 - a) cria um banco de dados hipotético de 1000 entradas (um vetor[1000] e
 - b) um cache de 100 entradas, e que
 - c) lê as entradas do arquivo `log` ciclicamente, em loop infinito.
 - d) implementa a estratégias *Cache-aside caching*
 - e) e trata o sinal Cntl-Z, e nesse caso limpa todo o cache, mas continua acessando o banco de dados.
- 3) faça o mesmo com a politica *Cache write through*
- 4) Discuta as diferenças em termos de em termos de tempo de processamento

Perguntas?

